Quantum Software Mini-Project

1081502

Hilary Term 2024

1

a

The routine will decompose Clifford+T diagrams into a sum of classically computable stabiliser terms - 'Clifford' diagrams - building off of previous work [1, 2]. Clifford+T diagrams can contain T-gates which make them more difficult to classically simulate, as opposed to Clifford, which are efficient to simulate classically. Given t T-gates (T-count), we can use identities found in previous work to obtain a sum of $2^{\alpha t}$ stabiliser terms. The number of these terms is the dominant contributor to the efficiency of the decomposition. The aim is to minimise the value of α, and this work improves upon [1].

b

Previous work [1] uses a greedy approach to obtaining the best vertex cuts for efficient decomposition, based on a heuristic in the paper. It also itself builds off the Brayvi, Smith, and Smolin (BSS) decomposition [3] used and shown in PyZX [4]. This work improves the previous work by treating it as a search problem, and attempts to find a possibly more optimal cut based on the existing procedure.

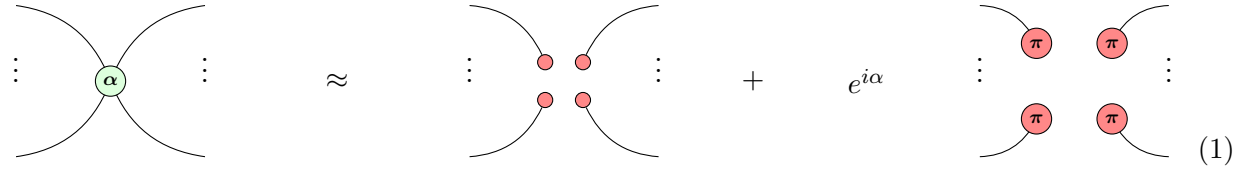
In short, obtaining the heuristic can be described as follows [1]:

1. Start at tier $t = 0$.
2. Assign weights at tier t to CNOT vertices which separate T-like. CNOT vertices involved in separating separating less T-like spiders are given higher weights.
3. Proceed to the next tier $t \leftarrow t+1$, and similar to step 2, for any weighted previous tier vertex which is separated from fusing with lower tier weighted vertices, by CNOTs, add weights to the CNOT vertices.
4. Repeat until no weightings are found at the largest tier $t = t_{max}$.
5. Select among the vertices that has a positive weight in the highest tier, the one whose maxmium across all tier is the largest. This will be the cut vertex.

Some details are omitted for brevity.

The subsequent procedure (ProcCut) uses the heuristics to carry out simplification without compromising the graph structure and is described fully in [1].

A key part of the procedure is using the following spider decomposition to take advantage of the graph structure.



(1)

This work then makes use of the same heuristic and also considers other vertices among the highest tier to cut. It uses Monte Carlo tree search (MCTS) to determine the strength of each vertex cut considered, simulating the final T-count (MCTS ProcCut). MCTS has 4 main steps in each iteration [5]:

1. Selection: From the root R, select child nodes which are more promising until a leaf node L
2. Expansion: If L is not terminal (T-count = 0), then expand the node and choose node C
3. Simulation: Randomly playout from C until we reach a terminal node
4. Backpropogation: Use the result of the simluation to update nodes from C to R.

The MCTS setup is shown in Appendix A. In the case of Clifford+T diagram decomposition, a node can be described by a sum of graphs that need to be decomposed. As a result, the final decomposition chosen will pick out the best sequence of decompositions that minimises the final number of stabiliser terms and thus, effective α.

This simple modification allows the algorithm to find cuts that are slight more optimal, especially in the specific type of circuits used to benchmark [1].

The algorithm below shows the function changed to obtain the list of possible vertices v_{poss}, rather than the best vertex $v_{best} = \max_{w(v)} v_{poss}$, where $w(v)$ is the weight of vertex v.

Algorithm 1 bestCut

Require: g: GraphS, $vweights\_max$: list[int], $vtiers$: list[int]
Ensure: list of possible vertices

```
maxTier ← max(vtiers)
List of possible vertices possV ← []
for i in range(len(vweights_max)) do
    w ← vweights_max[i]
    if w ≤ 0 or vtiers[i] ≠ maxTier then
        continue
    end if
    possV.append(i)
    if g.phase(i) ∈ {0.25, 0.75, 1.25, 1.75} then
        w ← w + 1          ▷ Add 1 to weight if the prospective vertex to cut is itself T-like
    end if
end for
return possV
```

c

Using the random circuit generation in [1], we can benchmark in a similar manner. MCTS ProcCut works well, especially if enough iterations of MCTS are allowed for leaf nodes to be explored. It will always match or outperform the original algorithm. The drawback, is of course, the complexity which is discussed in the last section.

Using the PyZX library [4] function `zx.generate.cliffordT()`, we can also generate random circuits, varying the `qubit_amount` and `gate_count`, and making sure more T-gates and CNOT gates appear with `p_t` and `p_cnot`. The details using this are described in the last section.

MCTS ProcCut working well for these circuits is to be expected as it is an extension of ProcCut [1] which takes advantage of the structure of T-gates being sandwiched between CNOT gates, but simply considers more possible vertex cuts.

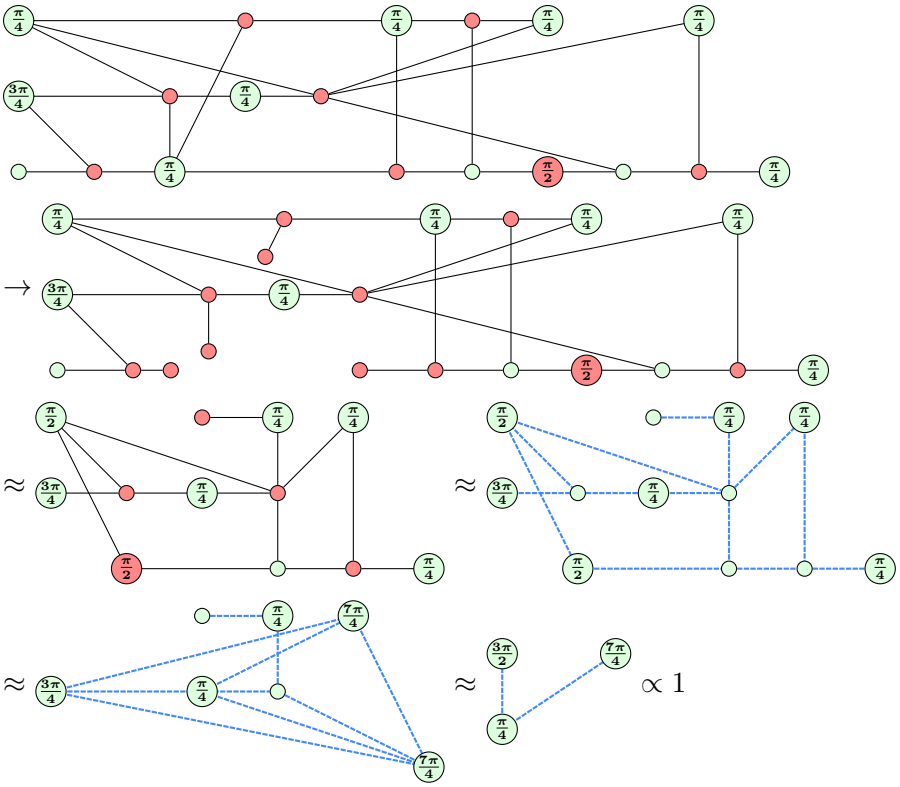
d

The routine will always work since it falls back to BSS decomposition, similar to the original procedure. However, it can be noted that it does have a limit of size and depth of circuit due to the exponential growth of both time and space, that matches the previous work [1]. The next section discusses the updated time and space complexity when using MCTS.

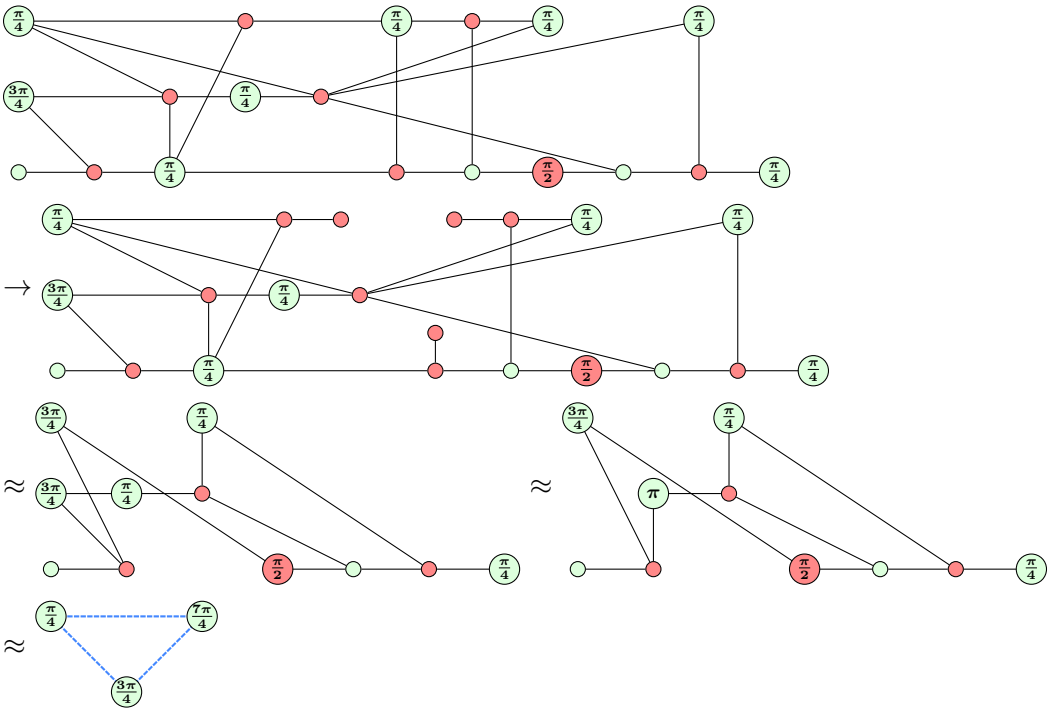
e

As an example of where the routine would shine, below is an example of a ZX-diagram where the greedy heuristic is not actually the optimal cut. In the examples shown below, the graph of the left term obtained when using Equation 1 is simplified using the ZX rewrite rules.

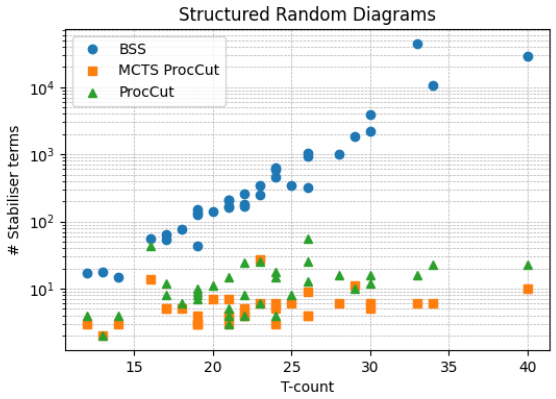
Here, the updated algorithm is able to immediately break decompose the graph in a single cut. Some steps are omitted for brevity - PyZX's `zx.simplify.full_reduce()` was used to check the outcome.

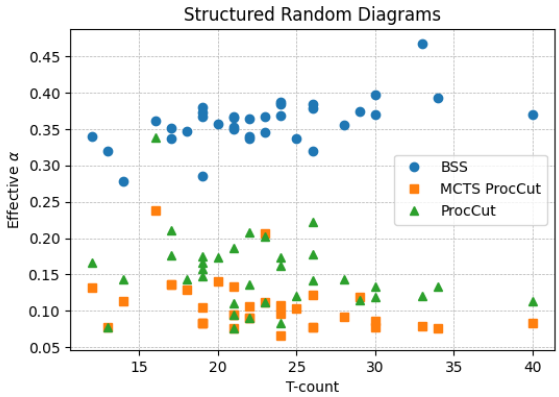


On the other hand, the original algorithm selects a suboptimal vertex cut.

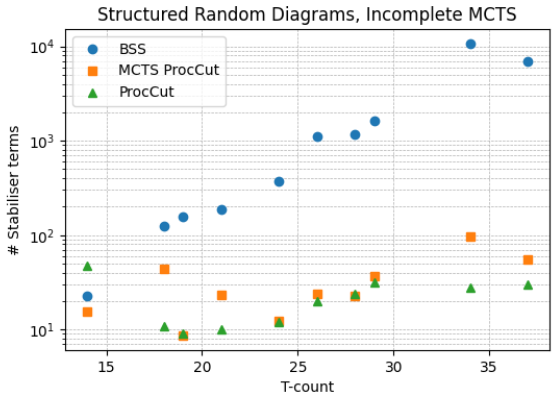


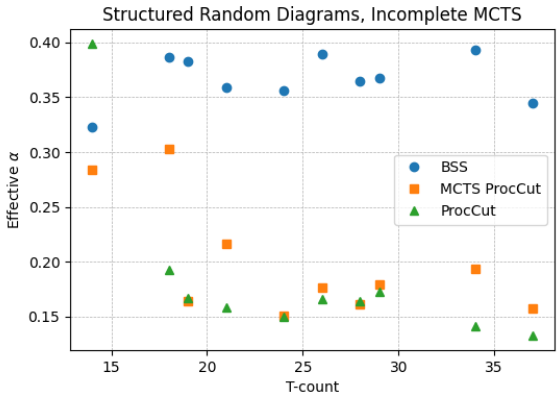
We can analyse the actual performance of the algorithm using a similar benchmark as in [1]. We then see that for the types of circuits described made of CNOTs, $\frac{n\pi}{4}$ phase gates, and Toffolis, when MCTS is able to explore leaf nodes, the number of stabiliser terms is reduced.





A good selection of the number of iterations for MCTS and number of children considered at each node will allow for more optimal values of α. On the other hand, when MCTS is not able to complete due to not having enough iterations, its performance can be unreliable.





The same algorithm was also tested using `zx.generate.cliffordT()` in PyZX, where the benefit over BSS was not as clear as on the highly structured diagrams. However, there are clear performance boosts over the original algorithm if MCTS is able to complete exploring until the leaf nodes. The configuration used, and their results are in the appendix A.

Analysing the complexity, if the runtime of the original algorithm is C, the MCTS makes it $O(Cvk)$, where v is the number of random children, and k is the number of iterations. Keeping v low means that the change in runtime can be kept minimal while keeping the potential for finding the optimal cut. Unless $P = NP$, we have that the C will grow exponentially [6], so no polynomial time solution is expected. We find that the MCTS approach allows for flexibility between choosing a high number of iterations k, versus a high number of random children v. A high number of iterations will allow the algorithm to obtain the most optimal solution it can find, since being able to explore until a leaf will mean it relies on BSS less, especially on the more structure diagrams. A high number of random children will make it more likely it does not miss a more optimal solution.

Given the memory usage of the original algorithm to be M, the requirement to keep the search space grows with $O(Mv)$ with the current implementation. This is a tradeoff, as it means it will run out of memory earlier than the original algorithm.

In conclusion, the addition of MCTS improves the base algorithm, with the tradeoff of having higher time and space complexity, but allows for flexible configurations, and works very well with the more structured diagrams.

A

Algorithm 2 Monte Carlo Tree Search

Require: *root*: Node, *iterations*: int = 300

1: **for** i in range($iterations$) **do**
2: $leaf \leftarrow$ selection($root$)
3: $simulation\_result \leftarrow$ simulation($leaf$)
4: backpropagation($leaf$, $simulation\_result$)
5: **end for**

```python
import numpy as np
from itertools import product

MAX_ACTIONS = 3

class Node:
    def __init__(self, graphs: list[GraphS], parent=None, vertex_cut_list: list[int]=None,
                 cliffCount: int=None, scalar: complex=None, depth:int=None, max_actions:int=None):
        self.graphs = graphs # The current state of the ZX-diagram
        self.parent = parent # Parent node
        self.vertex_cut_list = vertex_cut_list # Vertices cut to reach this node
        self.children = [] # List of child nodes
        self.score = 0.0 # Number of terms
        self.visits = 0 # Number of visits during simulations
        self.actions_done = set()
        self.max_actions = max_actions or MAX_ACTIONS
        self.scalar = scalar or complex(0)
        self.cliffCount = cliffCount or 0
        self.depth = depth or 0
        self.untried_actions = None
        self.best_action = None
        self.all_actions = None

    def __len__(self):
        return len(self.graphs)

    def is_fully_expanded(self):
        # Assuming a function that returns all possible cuts
        if self.is_terminal(): return True
        untried_actions = self.get_untried_actions()
        if not untried_actions:
            return len(self.children) >= 1
        return len(self.children) >= self.max_actions

    def is_terminal(self):
        return not self.graphs

    def best_child(self, c_param=1.4):
        # Using UCB
        choices_weights = [
            (child.score / child.visits) + c_param * np.sqrt((2 * np.log(self.visits) / child.visits))
            for child in self.children
        ]
        return self.children[np.argmax(choices_weights)]

    def get_all_actions(self):
        if self.all_actions is not None: return self.all_actions
        vList = []
        bList = []
        for g in self.graphs:
            g = g.copy()

            vBest,tier,vweights_max,vweights,vtiers, possV = compWeightsM(g,False)
```

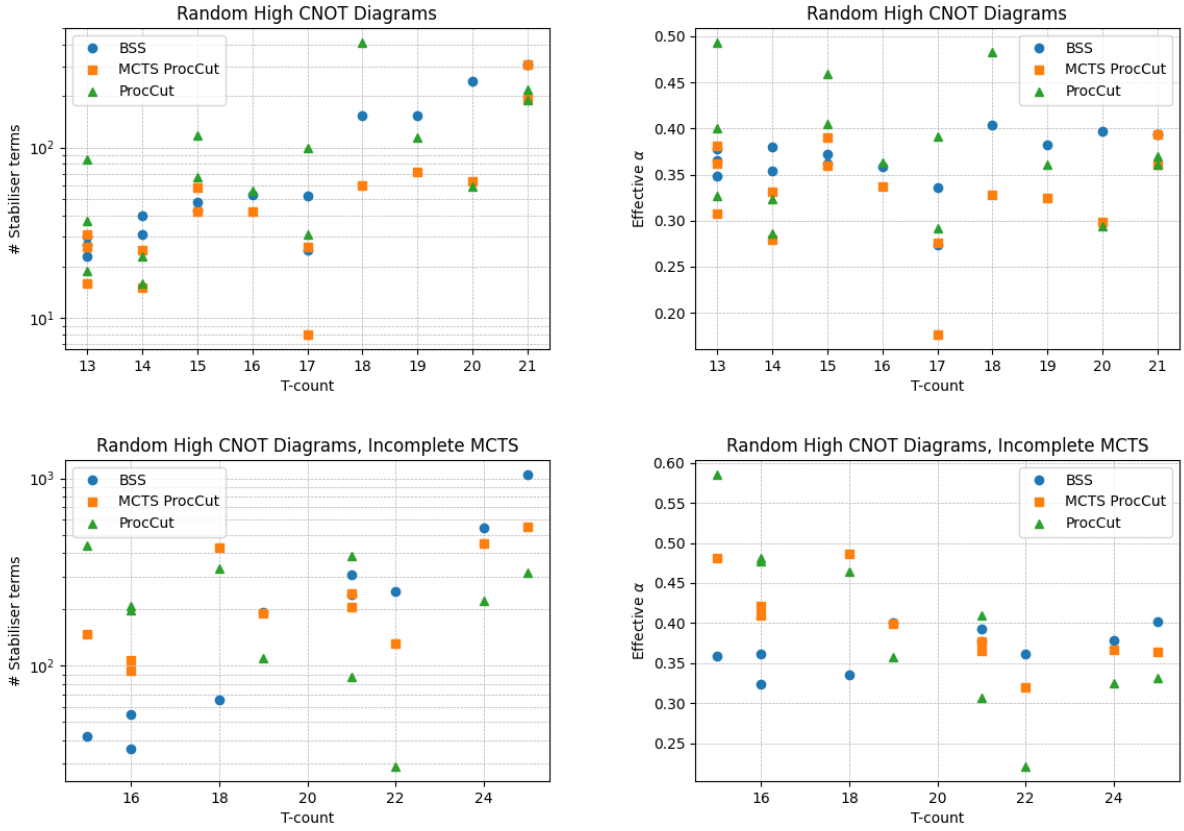
```
            orderedV = sorted([(w, v) for v, w in enumerate(vweights_max) if w > 0], reverse=True)
            possV = [v for w, v in orderedV]
            vList.append(tuple(possV[:self.max_vertices]))
            bList.append(vBest)

        self.best_action = tuple(bList)
        self.all_actions = set(product(*vList))
        self.untried_actions = self.all_actions.copy()
        return self.all_actions

    def get_untried_actions(self):
        if self.untried_actions is not None: return self.untried_actions
        return self.get_all_actions()

    def get_best_action(self):
        if self.best_action is not None: return self.best_action
        self.get_all_actions()
        return self.best_action
```

`zx.simplify.full_reduce()` is used with `qubit_amount` varying from 6 to 8, and `gate_count` varying from 50 to 150. The probability of T-gates and CNOTs were chosen at `p_t = 0.4` and `p_cnot = 0.4`.



References

[1] Matthew Sutcliffe and Aleks Kissinger. Procedurally Optimised ZX-Diagram Cutting for Efficient T-Decomposition in Classical Simulation. *arXiv preprint arXiv:2403.10964*, 2024.

[2] Aleks Kissinger and John van de Wetering. Simulating quantum circuits with zx-calculus reduced stabiliser decompositions. *Quantum Science and Technology*, 7(4):044001, July 2022.

[3] Sergey Bravyi, Graeme Smith, and John A. Smolin. Trading classical and quantum computational resources. *Phys. Rev. X*, 6:021043, Jun 2016.

[4] Aleks Kissinger and John van de Wetering. PyZX: Large Scale Automated Diagrammatic Reasoning. In Bob Coecke and Matthew Leifer, editors, Proceedings 16th International Conference on *Quantum Physics and Logic,* Chapman University, Orange, CA, USA., 10-14 June 2019, volume 318 of *Electronic Proceedings in Theoretical Computer Science*, pages 229–241. Open Publishing Association, 2020.

[5] Guillaume Chaslot, Mark Winands, H. Jaap Van Den Herik, Jos Uiterwijk, and Bruno Bouzy. Progressive strategies for monte-carlo tree search. *New Mathematics and Natural Computation*, 04:343–357, 11 2008.

[6] Sergey Bravyi, Dan Browne, Padraic Calpin, Earl Campbell, David Gosset, and Mark Howard. Simulation of quantum circuits by low-rank stabilizer decompositions. *Quantum*, 3:181, September 2019.